

Movie Recommendation System - Kế hoạch triển khai trong 2 tuần

Dataset sử dụng

- `movies.csv` khoảng 3.04 MB: gồm `movieId`, `title`, `genres`.
- `ratings.csv` khoảng 678.26 MB: gồm `userId`, `movieId`, `rating`, `timestamp`.

Nguồn tham khảo

- Kaggle dataset: [Movie Recommendation System](#).
- PDF bài học: `Documents_2026-4_M12W02 - Recommendation Systems_[Description_v1]-Recommendation-System.pdf`.

Scope đã chốt: chỉ dùng 3 phương pháp:

- Simple Recommender
- Content-based Recommender
- Collaborative Filtering

Không triển khai Hybrid Recommender, Context-aware Recommender, Matrix Factorization nâng cao hoặc Neural CF trong phạm vi 2 tuần này.

1. Mục tiêu dự án

Dự án xây dựng một hệ thống gợi ý phim dựa trên dữ liệu phim và dữ liệu rating của user.

Hệ thống cần trả lời được 3 nhu cầu chính:

1. User mới nên xem phim nào?

- Dùng **Simple Recommender**.
 - Gợi ý các phim phổ biến, được nhiều người đánh giá cao.
2. Nếu thích một phim cụ thể, nên xem phim nào tương tự?

- Dùng **Content-based Recommender**.
 - Gợi ý phim có thể loại gần giống phim đầu vào.
3. Một user cụ thể có thể thích phim nào nhưng chưa xem/chưa rating?

- Dùng **Collaborative Filtering**.
 - Gợi ý dựa trên hành vi rating của cộng đồng user.

Ví dụ output mong muốn:

Rank	movieId	title	genres	score	method
1	318	Shawshank Redemption, The (1994)	Crime Drama	0.94	Collaborative Filtering
2	296	Pulp Fiction (1994)	Comedy Crime Drama Thriller	0.91	Collaborative Filtering
3	356	Forrest Gump (1994)	Comedy Drama Romance War	0.88	Collaborative Filtering

2. Dataset

2.1. `movies.csv`

File này mô tả thông tin cơ bản của phim.

Cột	Kiểu dữ liệu	Ý nghĩa	Ví dụ
<code>movieId</code>	integer	ID duy nhất của phim	1
<code>title</code>	string	Tên phim, thường kèm năm phát hành	Toy Story (1995)
<code>genres</code>	string	Các thể loại phim, phân tách bằng `	`

Vai trò trong project:

- Dùng để hiển thị tên phim và thể loại.

- Dùng để làm **Content-based Recommender**.
- Dùng để join với `ratings.csv` thông qua `movieId`.

2.2. ratings.csv

File này chứa lịch sử đánh giá phim của user.

Cột	Kiểu dữ liệu	Ý nghĩa	Ví dụ
<code>userId</code>	integer	ID của user	1
<code>movieId</code>	integer	ID phim được user đánh giá	296
<code>rating</code>	float	Điểm đánh giá, thường từ 0.5 đến 5.0	5.0
<code>timestamp</code>	integer	Thời điểm user rating, dạng Unix timestamp	1147880044

Vai trò trong project:

- Dùng để tính phim phổ biến/chất lượng cao.
- Dùng để xây **Utility Matrix** cho Collaborative Filtering.
- Dùng để train/test split theo thời gian.

[!important] Lưu ý kỹ thuật `ratings.csv` khoảng 678 MB, khá lớn. Khi xử lý cần dùng dtype tiết kiệm RAM và sparse matrix. Không nên tạo pivot table dense toàn bộ user x movie vì dễ làm đầy RAM.

3. Ba phương pháp sẽ triển khai

3.1. Simple Recommender

Mục tiêu

Simple Recommender gợi ý cùng một danh sách phim tốt cho mọi user.

Phương pháp này phù hợp khi:

- User mới chưa có lịch sử rating.
- Hệ thống cần trang "Top movies".
- Cần một baseline đơn giản để so sánh với các model khác.

Dữ liệu sử dụng

Simple Recommender dùng:

- `ratings.csv` : để tính điểm trung bình và số lượt rating.
- `movies.csv` : để lấy tên phim và thể loại.

Cách làm

Bước 1: Gom rating theo từng phim.

```
movieId
-> avg_rating
-> num_ratings
```

Bước 2: Tính điểm gợi ý.

Không nên chỉ dùng `avg_rating`, vì một phim có rating 5.0 nhưng chỉ 1 lượt đánh giá không đáng tin bằng phim rating 4.5 với 10.000 lượt đánh giá.

Do đó dùng **weighted rating**.

Công thức `weighted_rating` dựa trên công thức nào?

Công thức này thường được biết đến dưới tên **IMDb weighted rating formula**. Về bản chất, nó là một dạng **Bayesian average**.

Hiểu đơn giản:

- Nếu một phim có **rất nhiều lượt rating**, ta tin điểm trung bình riêng của phim đó hơn.
- Nếu một phim có **rất ít lượt rating**, ta chưa nên tin hoàn toàn vào điểm riêng của nó.
- Khi chưa đủ tin, điểm của phim sẽ bị kéo gần về điểm trung bình chung của toàn bộ dataset.

Vì vậy công thức này trộn 2 nguồn thông tin:

Thành phần	Ý nghĩa
R	Điểm trung bình riêng của phim
C	Điểm trung bình chung của toàn bộ dataset

Tỷ lệ trộn phụ thuộc vào `v` , tức số lượt rating của phim.

Nếu `v` lớn:

```
v / (v + m) gần 1
m / (v + m) gần 0
```

Khi đó `WR` gần với `R` , vì phim có nhiều lượt rating nên điểm riêng đáng tin.

Nếu `v` nhỏ:

```
v / (v + m) gần 0
m / (v + m) gần 1
```

Khi đó `WR` gần với `C` , vì phim có quá ít lượt rating nên ta kéo điểm về mức trung bình chung.

Công thức:

```
WR = (v / (v + m)) * R + (m / (v + m)) * C
```

Trong đó:

- `R` : rating trung bình của phim.
- `v` : số lượt rating của phim.
- `m` : số lượt rating tối thiểu để phim được xem là đủ tin cậy.
- `C` : rating trung bình của toàn bộ dataset.

Tại sao dùng công thức này trong project?

Trong project này, `ratings.csv` có rất nhiều phim và số lượt rating giữa các phim không đều nhau. Có phim rất nổi tiếng nên có hàng nghìn hoặc hàng chục nghìn lượt rating. Cũng có phim chỉ có vài lượt rating.

Nếu chỉ sắp xếp theo `avg_rating` , hệ thống sẽ dễ đưa các phim ít người rating lên top chỉ vì vài rating đầu tiên quá cao. Điều đó làm Simple Recommender thiếu ổn định.

`weighted_rating` giúp Simple Recommender chuyên nghiệp hơn vì:

- Giảm bias cho phim có quá ít rating.
- Ưu tiên phim vừa được đánh giá cao vừa có nhiều lượt đánh giá.
- Phù hợp cho tab "Top movies" hoặc fallback khi user mới chưa có lịch sử rating.
- Tạo baseline công bằng hơn để so sánh với Content-based Recommender và Collaborative Filtering.

Ví dụ:

Phim	R rating trung bình	v số lượt rating	Nhận xét
A	5.0	2	Điểm cao nhưng chưa đáng tin
B	4.6	20.000	Điểm thấp hơn một chút nhưng đáng tin hơn

Với `weighted_rating` , phim A sẽ bị kéo điểm về gần trung bình chung `C` , còn phim B gần như giữ nguyên điểm 4.6 vì có nhiều lượt đánh giá.

Bước 3: Sắp xếp phim theo `weighted_rating` giảm dần.

Output

Input:
top_k = 10

Output:
Top 10 phim có weighted_rating cao nhất.

Ưu điểm

- Dễ triển khai.
- Chạy nhanh.
- Không cần biết user là ai.
- Là fallback tốt khi user mới chưa có dữ liệu.

Hạn chế

- Không cá nhân hóa.
- Mọi user gần như nhận cùng một danh sách.
- Không tận dụng được sở thích riêng của từng user.

Test case

Test	Kỳ vọng
top_k = 10	Trả đúng 10 phim
Phim ít rating nhưng điểm cao	Không đứng quá cao nếu num_ratings thấp
Kết quả trả về	Có đủ movieId, title, genres, avg_rating, num_ratings, weighted_rating

Nhóm cần đánh giá gì ở phương pháp này?

Tiêu chí	Câu hỏi cần trả lời	Cách kiểm tra
Độ hợp lý của top phim	Top phim có thật sự là phim được nhiều người đánh giá cao không?	Xem avg_rating, num_ratings, weighted_rating
Độ tin cậy	Có phim rất ít rating nhưng vẫn lọt top không?	Kiểm tra các phim top có num_ratings đủ lớn không
Tính hữu ích	Danh sách này có dùng được cho user mới không?	Demo tab "Top movies" khi chưa nhập userId
Tốc độ	Có trả kết quả nhanh không?	Đo thời gian sort/top-K sau khi đã tính sẵn stats
Vai trò trong hệ thống	Có nên dùng làm model chính không?	Không nên. Dùng làm baseline và fallback

Kết luận đánh giá

Simple Recommender là phương pháp dễ làm nhất và bắt buộc nên có. Nó không cá nhân hóa, nhưng rất hữu ích để:

- Làm baseline ban đầu.
- Gợi ý cho user mới.
- Kiểm tra pipeline đọc và xử lý ratings.csv.
- So sánh với Content-based Recommender và Collaborative Filtering.

3.2. Content-based Recommender

Mục tiêu

Content-based Recommender gợi ý phim tương tự với một phim đầu vào.

Trong dataset này, movies.csv chỉ có genres làm metadata chính. Vì vậy content-based sẽ dựa trên thể loại phim.

Ví dụ:

Input: Toy Story (1995)
Genres: Adventure|Animation|Children|Comedy|Fantasy

Output kỳ vọng:
Các phim có nhiều genre giống Toy Story,
ví dụ Animation, Children, Comedy, Fantasy.

Dữ liệu sử dụng

Content-based Recommender dùng:

- `movies.csv`.
- Cột chính: `movieId`, `title`, `genres`.

Không cần dùng `ratings.csv` cho phương pháp này.

Cách làm

Bước 1: Tách `genres`.

```
"Adventure|Animation|Children|Comedy|Fantasy"  
-> ["Adventure", "Animation", "Children", "Comedy", "Fantasy"]
```

Bước 2: Biến genre thành vector.

Có thể dùng one-hot encoding:

movieId	Adventure	Animation	Comedy	Drama	Thriller
1	1	1	1	0	0
2	1	0	0	0	0
3	0	0	1	0	0

Hoặc dùng `CountVectorizer` / `TfidfVectorizer` trên chuỗi `genres_text`.

Bước 3: Tính độ giống nhau giữa các phim.

```
similarity(movie_a, movie_b) = cosine(movie_a_vector, movie_b_vector)
```

Bước 4: Khi user nhập phim, lấy top-K phim có cosine similarity cao nhất.

Output

Input:
`movieId = 1` hoặc `title = "Toy Story (1995)"`
`top_k = 10`

Output:
Top 10 phim có genre giống phim đầu vào nhất.

Ưu điểm

- Dễ giải thích.
- Không cần lịch sử rating.
- Có thể gợi ý cho phim mới nếu phim đó có genre.
- Chạy nhanh vì `movies.csv` nhỏ.

Hạn chế

- Chỉ dựa vào genre nên kết quả còn thô.
- Hai phim cùng genre nhưng nội dung khác xa vẫn có thể được xem là giống nhau.
- Không học được sở thích riêng của user.

Test case

Input	Kỳ vọng
Toy Story (1995)	Gợi ý nhiều phim có Animation, Children, Comedy, Fantasy
Heat (1995)	Gợi ý phim có Action, Crime, Thriller
Movie không tồn tại	Báo lỗi rõ ràng hoặc gợi ý title gần đúng
Top-K result	Không trả lại chính phim đầu vào

Nhóm cần đánh giá gì ở phương pháp này?

Tiêu chí	Câu hỏi cần trả lời	Cách kiểm tra
Độ giống về genre	Các phim được gợi ý có cùng thể loại với phim đầu vào không?	Tính Genre overlap@K
Khả năng giải thích	Có giải thích được vì sao phim được gợi ý không?	Hiển thị các genre trùng nhau
Tính ổn định	Với cùng một phim input, kết quả có nhất quán không?	Chạy lại nhiều lần, kết quả không đổi
Xử lý lỗi input	Nếu nhập sai title thì app có báo lỗi rõ không?	Test title không tồn tại hoặc title gần giống
Vai trò trong hệ thống	Có dùng được khi không có rating không?	Có. Chỉ cần movies.csv

Kết luận đánh giá

Content-based Recommender là phương pháp quan trọng thứ hai sau Simple Recommender. Nó phù hợp để gợi ý phim tương tự theo nội dung, đặc biệt khi:

- User chọn một phim cụ thể.
- User chưa có lịch sử rating.
- Nhóm cần một kết quả dễ giải thích trong demo.

Tuy nhiên, vì dataset chỉ có genres, chất lượng gợi ý sẽ chưa sâu. Nhóm cần ghi rõ hạn chế này trong báo cáo để tránh kỳ vọng quá cao.

3.3. Collaborative Filtering

Mục tiêu

Collaborative Filtering gợi ý phim dựa trên hành vi rating của nhiều user.

Ý tưởng cốt lõi:

Nếu nhiều user cùng thích hai phim A và B, thì A và B có thể liên quan đến nhau. Nếu một user thích phim A nhưng chưa xem phim B, hệ thống có thể gợi ý phim B cho user đó.

Phương pháp này giúp hệ thống cá nhân hóa kết quả tốt hơn Simple và Content-based.

Dữ liệu sử dụng

Collaborative Filtering dùng:

- ratings.csv: userId, movieId, rating, timestamp.
- movies.csv: để map movieId sang title và genres.

Utility Matrix

Theo PDF, Collaborative Filtering thường bắt đầu từ **Utility Matrix**.

Utility Matrix là bảng user x movie:

userId	Toy Story	Jumanji	Heat

1	?	?	5.0
userId	Toy Story	Jumanji	Heat
2	4.0	3.5	?
3	5.0	?	4.0

Ý nghĩa:

- Mỗi hàng là một user.
- Mỗi cột là một movie.
- Mỗi ô là rating user dành cho movie.
- Dấu ? nghĩa là user chưa rating phim đó.

Vì user thường chỉ rating một phần rất nhỏ trong toàn bộ phim, Utility Matrix rất thưa. Do đó cần dùng `scipy.sparse.csr_matrix`.

Hướng triển khai trong 2 tuần

Trong scope 2 tuần, nên chọn **Item-based Collaborative Filtering** làm hướng chính.

Lý do:

- Dễ triển khai hơn deep learning.
- Ổn định hơn User-based CF khi số user lớn.
- Phù hợp với dataset có nhiều rating.
- Dễ giải thích khi demo.

Pipeline:

```
ratings.csv
-> lọc user/movie đủ rating
-> tạo sparse Utility Matrix: user x movie
-> transpose thành item x user
-> tính item-item cosine similarity
-> với user mục tiêu:
    lấy các phim user đã rating cao
    tìm các phim tương tự
    loại phim user đã rating
    trả top-K phim gợi ý
```

Cách tạo recommendation cho một user

Ví dụ user 1 đã rating cao:

```
Heat (1995): 5.0
Pulp Fiction (1994): 5.0
Fargo (1996): 4.5
```

Hệ thống sẽ:

1. Lấy các phim user đã thích, ví dụ rating ≥ 4.0 .
2. Tìm các phim tương tự với từng phim đó bằng item-item similarity.
3. Cộng/trung bình score similarity.
4. Loại bỏ phim user đã rating.
5. Trả top-K phim còn lại.

Output

```
Input:
userId = 1
top_k = 10

Output:
Top 10 phim user 1 có thể thích nhưng chưa rating.
```

Ưu điểm

- Có cá nhân hóa.
- Khai thác được dữ liệu rating của cộng đồng.
- Có thể gợi ý phim khác genre nhưng cùng pattern sở thích.
- Phù hợp với bài học Collaborative Filtering trong PDF.

Hạn chế

- Cần nhiều rating.
- User mới không có lịch sử rating sẽ khó gợi ý.
- Movie mới chưa có rating cũng khó được gợi ý.
- Nếu tạo ma trận dense, hệ thống dễ bị quá tải RAM.

Test case

Test	Kỳ vọng
<code>userId</code> tồn tại	Trả top-K phim user chưa rating
<code>userId</code> không tồn tại	Fallback sang Simple Recommender
User đã rating nhiều phim	Kết quả không chứa phim đã rating
Dataset lớn	Pipeline dùng sparse matrix, không crash RAM

Nhóm cần đánh giá gì ở phương pháp này?

Tiêu chí	Câu hỏi cần trả lời	Cách kiểm tra
Cá nhân hóa	Kết quả có khác nhau giữa các user không?	So sánh top-K của nhiều <code>userId</code>
Không lặp phim cũ	Model có gợi ý lại phim user đã rating không?	Join kết quả với lịch sử rating của user
Chất lượng ranking	Phim test có xuất hiện trong top-K không?	Tính <code>HR@10</code>
Vị trí phim đúng	Nếu phim đúng xuất hiện, nó nằm gần top đầu không?	Tính <code>NDCG@10</code>
Hiệu năng	Có chạy được với <code>ratings.csv</code> lớn không?	Dùng sparse matrix và test trên sample trước
Khả năng fallback	User mới không có rating thì xử lý thế nào?	Chuyển sang Simple Recommender

Kết luận đánh giá

Collaborative Filtering là phương pháp quan trọng nhất nếu mục tiêu là gợi ý cá nhân hóa. Đây là phần cần nhiều công nhất vì phải xử lý `ratings.csv` , tạo Utility Matrix và đánh giá bằng metric.

Trong 2 tuần, nhóm nên ưu tiên **Item-based Collaborative Filtering** thay vì model deep learning vì:

- Dễ giải thích.
- Dễ debug.
- Phù hợp với dữ liệu rating hiện có.
- Có thể đánh giá bằng `HR@10` và `NDCG@10` .
- Khả thi hơn với thời gian và nguồn lực của nhóm.

3.4. Bảng so sánh để cả nhóm đánh giá

Tiêu chí	Simple Recommender	Content-based Recommender	Collaborative Filtering
Input chính	Không cần user/movie cụ thể	<code>movieId</code> hoặc <code>title</code>	<code>userId</code>
Dữ liệu dùng	<code>ratings.csv</code> + <code>movies.csv</code>	<code>movies.csv</code>	<code>ratings.csv</code> + <code>movies.csv</code>
Có cá nhân hóa không?	Không	Không theo user, chỉ theo phim input	Có
Xử lý user mới	Tốt	Tốt nếu user chọn phim	Không tốt, cần fallback
Xử lý movie mới	Cần có rating mới rank tốt	Tốt nếu có genre	Không tốt nếu chưa có rating

Tiêu chí Độ khó triển khai	Simple Recommender Thấp	Content-based Recommender Trung bình thấp	Collaborative Filtering Trung bình cao
Tốc độ chạy	Rất nhanh	Nhanh	Phụ thuộc sparse matrix và số item
Khả năng giải thích	Dễ: rating cao, nhiều lượt rating	Dễ: cùng genre	Trung bình: user tương tự hoặc item tương tự
Metric chính	weighted_rating, sanity check	Genre overlap@K	HR@10, NDCG@10
Vai trò trong project	Baseline + fallback	Similar movie recommendation	Personalized recommendation

3.5. Thứ tự ưu tiên triển khai

Nhóm nên làm theo thứ tự sau:

```
1. Simple Recommender
-> nhanh có kết quả
-> kiểm tra được ratings.csv
-> làm fallback cho user mới

2. Content-based Recommender
-> dùng movies.csv
-> demo được chức năng tìm phim tương tự
-> dễ giải thích cho báo cáo

3. Collaborative Filtering
-> dùng ratings.csv
-> tạo gợi ý cá nhân hóa
-> cần nhiều thời gian nhất để xử lý sparse matrix và evaluation
```

Nếu đến cuối tuần 1 mà Simple và Content-based chưa ổn, không nên vội làm Collaborative Filtering toàn bộ dataset. Khi đó nên làm CF trên sample trước để đảm bảo có demo chạy được.

4. Pipeline tổng thể

Pipeline sẽ đi theo thứ tự sau:

```
1. Load data
movies.csv
ratings.csv

2. Clean data
movies: xử lý title, year, genres
ratings: xử lý dtype, missing, duplicate

3. Build Simple Recommender
ratings -> avg_rating, num_ratings, weighted_rating

4. Build Content-based Recommender
movies.genres -> genre vectors -> cosine similarity

5. Build Collaborative Filtering
ratings -> sparse utility matrix -> item-item similarity

6. Evaluation
Simple: kiểm tra top phim phổ biến
Content-based: kiểm tra genre overlap
Collaborative Filtering: HR@10, NDCG@10 hoặc manual test trên user sample

7. Serving
Streamlit hoặc FastAPI
```

5. Data Processing chi tiết

5.1. Làm sạch `movies.csv`

Các bước:

- 1. Đọc file `movies.csv`.
- 2. Kiểm tra `movieId` có bị trùng không.
- 3. Tách năm phát hành từ `title`.
- 4. Tách `genres` theo dấu `|`.
- 5. Xử lý (`no genres listed`) nếu có.
- 6. Tạo cột `genres_text`.

Output:

```
movies_clean.parquet
```

Schema gợi ý:

Cột	Ý nghĩa
<code>movieId</code>	ID phim
<code>title</code>	Tên phim
<code>year</code>	Năm phát hành
<code>genres</code>	Genre gốc
<code>genres_list</code>	List genre
<code>genres_text</code>	Genre đã nối thành text

5.2. Làm sạch `ratings.csv`

Các bước:

- 1. Chỉ đọc 4 cột cần thiết: `userId`, `movieId`, `rating`, `timestamp`.
- 2. Dùng `dtype` tiết kiệm RAM:

```
dtype = {
    "userId": "int32",
    "movieId": "int32",
    "rating": "float32",
    "timestamp": "int64",
}
```

- 3. Xóa duplicate nếu có.
- 4. Lọc `rating` không hợp lệ nếu có.
- 5. Lọc `user/movie` quá ít `rating` để model ổn định.

Rule đề xuất:

```
Giữ user có ít nhất 20 rating.
Giữ movie có ít nhất 50 rating.
```

Output:

```
ratings_clean.parquet
```

6. Evaluation

6.1. Simple Recommender

Đánh giá bằng kiểm tra logic:

- Phim có `weighted_rating` cao.
- Phim có đủ số lượt rating.
- Không để phim ít rating đứng top quá cao.

Output report:

Metric	Ý nghĩa
<code>avg_rating</code>	Rating trung bình
<code>num_ratings</code>	Số lượt rating
<code>weighted_rating</code>	Điểm cuối dùng để rank

6.2. Content-based Recommender

Đánh giá bằng:

Genre overlap@K

Công thức đơn giản:

Genre overlap@K = số phim trong top-K có ít nhất 1 genre trùng với phim input / K

Ví dụ:

Input: Toy Story (1995)
Top-K nên có nhiều phim trùng Animation, Children, Comedy hoặc Fantasy.

6.3. Collaborative Filtering

Đánh giá theo tinh thần PDF:

- **HR@10**: phim test của user có nằm trong top 10 không.
- **NDCG@10**: phim test nằm càng cao trong top 10 thì điểm càng tốt.

Cách split:

Với mỗi user:

- sắp xếp rating theo timestamp
- rating mới nhất làm test
- các rating trước đó làm train

Nếu dataset quá lớn và chưa kịp đánh giá toàn bộ, có thể evaluate trên sample user.

7. API/UI demo

Trong 2 tuần, nên dùng Streamlit để làm demo nhanh.

UI gồm 3 tab chính:

7.1. Tab 1 - Simple Recommender

Input:

- `top_k`
- optional: `genre`

Output:

- Top phim theo `weighted_rating`.

7.2. Tab 2 - Content-based Recommender

Input:

- `movie title` hoặc `movieId`
- `top_k`

Output:

- Top phim cùng/giống genre.
- Hiển thị lý do: các genre trùng nhau.

7.3. Tab 3 - Collaborative Filtering

Input:

- `userId`
- `top_k`

Output:

- Top phim user có thể thích.
- Không trả phim user đã rating.

8. Phân công nhóm 5 người

Vai trò	Trách nhiệm chính	Deliverable
Project Lead / App Integration	Chốt scope, quản lý timeline, ghép model vào Streamlit/API, chuẩn bị demo cuối	Demo app, slide/report, checklist tiến độ
AI Data	Tải dataset, EDA, làm sạch <code>movies.csv</code> và <code>ratings.csv</code> , tạo data dictionary	<code>movies_clean.parquet</code> , <code>ratings_clean.parquet</code> , EDA report
AI Pipeline	Xây pipeline đọc dữ liệu lớn, sparse matrix, lưu/load artifact	<code>pipeline.py</code> , <code>item_similarity.npz</code> , hướng dẫn chạy
AI Model	Xây 3 recommender: Simple, Content-based, Collaborative Filtering	<code>recommender.py</code> , model output, evaluation table
Tester	Viết test case, kiểm tra UI/API, kiểm tra top-K result, log bug	<code>test_cases.md</code> , test report

9. Timeline 2 tuần

Tuần 1 - Dữ liệu và baseline

Ngày	Công việc	Người phụ trách	Kết quả
1	Chốt scope chỉ gồm 3 phương pháp, tải dataset, xác nhận schema	Lead + AI Data	Scope và schema chính thức
2	EDA <code>movies.csv</code> : số phim, genre distribution, duplicate title	AI Data	EDA movies
3	EDA <code>ratings.csv</code> : số user, số rating, rating distribution, sparsity	AI Data + AI Pipeline	EDA ratings
4	Làm sạch dữ liệu, tối ưu dtype, lưu processed data	AI Data	<code>movies_clean</code> , <code>ratings_clean</code>
5	Xây Simple Recommender	AI Model	Top movies baseline
6	Xây Content-based Recommender theo genre	AI Model + AI Pipeline	Similar movies by genre
7	Demo Streamlit bản đầu với 2 tab Simple và Content-based	Lead + Tester	Demo v1 + bug list

Tuần 2 - Collaborative Filtering và hoàn thiện

Ngày	Công việc	Người phụ trách	Kết quả
8	Tạo sparse Utility Matrix	AI Pipeline	<code>utility_matrix.npz</code>
9	Tính item-item similarity cho Collaborative Filtering	AI Pipeline + AI Model	<code>item_similarity.npz</code>
10	Viết hàm recommend theo <code>userId</code> , loại phim đã rating	AI Model	Personalized recommendation
11	Đánh giá HR@10/NDCG@10 trên sample user	AI Model + Tester	Evaluation table
12	Thêm tab Collaborative Filtering vào UI, test edge cases	Lead + Tester	Demo v2
13	Tối ưu tốc độ, lưu/load artifacts, hoàn thiện test report	AI Pipeline + Tester	Demo ổn định
14	Viết báo cáo cuối, chuẩn bị slide, rehearsal demo	Cả nhóm	Bản nộp cuối

10. Backlog

Must-have

- Đọc được `movies.csv` và `ratings.csv`.
- Làm sạch dữ liệu.
- Có Simple Recommender.
- Có Content-based Recommender.
- Có Collaborative Filtering.
- Có Streamlit/API demo.
- Có test report.

Should-have

- HR@10 và NDCG@10 cho Collaborative Filtering.
- Fuzzy search title.
- Lưu/load sparse matrix và item similarity.
- Giải thích lý do gợi ý cho Content-based.

Won't-have trong 2 tuần

- Hybrid Recommender.
- Context-aware Recommender.
- Matrix Factorization/SVD.
- Neural Collaborative Filtering.
- A/B testing.
- Production deployment.

11. Rủi ro và cách xử lý

Rủi ro	Tác động	Cách xử lý
<code>ratings.csv</code> lớn khoảng 678 MB	Chậm hoặc hết RAM	Dùng dtype tối ưu, sample khi dev, dùng sparse matrix
Utility Matrix quá thưa	Similarity nhiều	Lọc user/movie quá ít rating
Tạo pivot table dense	Dễ crash RAM	Dùng <code>scipy.sparse.csr_matrix</code>
Title bị trùng hoặc nhập sai	Không tìm được phim	Dùng <code>movieId</code> làm khóa chính, thêm fuzzy search
User mới không có rating	CF không dùng được	Fallback sang Simple Recommender
Content-based chỉ có genre	Gợi ý còn thô	Nêu rõ hạn chế trong report

12. Cấu trúc thư mục đề xuất

```
ACM2/  
  document.md  
  data/  
    raw/  
      movies.csv  
      ratings.csv  
    processed/  
      movies_clean.parquet  
      ratings_clean.parquet  
  notebooks/  
    01_eda_movies.ipynb  
    02_eda_ratings.ipynb  
    03_modeling.ipynb  
  src/  
    data_processing.py  
    recommender_simple.py  
    recommender_content.py  
    recommender_cf.py  
    evaluation.py  
    app.py  
  artifacts/  
    genre_vectorizer.pkl  
    genre_similarity.npz  
    utility_matrix.npz  
    item_similarity.npz  
    movie_id_maps.pkl  
  tests/  
    test_cases.md  
    test_recommender.py  
  reports/  
    final_report.md
```

13. Pseudocode triển khai

13.1. Load data

```
import pandas as pd  
  
movies = pd.read_csv(  
    "data/raw/movies.csv",  
    dtype={"movieId": "int32", "title": "string", "genres": "string"}  
)  
  
ratings = pd.read_csv(  
    "data/raw/ratings.csv",  
    dtype={  
        "userId": "int32",  
        "movieId": "int32",  
        "rating": "float32",  
        "timestamp": "int64",  
    }  
)
```

13.2. Simple Recommender

```

movie_stats = (
    ratings
    .groupby("movieId")
    .agg(avg_rating=("rating", "mean"), num_ratings=("rating", "count"))
    .reset_index()
)

C = ratings["rating"].mean()
m = movie_stats["num_ratings"].quantile(0.80)

movie_stats["weighted_rating"] = (
    movie_stats["num_ratings"] / (movie_stats["num_ratings"] + m) * movie_stats["avg_rating"]
    + m / (movie_stats["num_ratings"] + m) * C
)

popular_movies = movie_stats.merge(movies, on="movieId")
popular_movies = popular_movies.sort_values("weighted_rating", ascending=False)

```

13.3. Content-based Recommender

```

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity

movies["genres_text"] = movies["genres"].str.replace("|", " ", regex=False)

vectorizer = CountVectorizer()
genre_matrix = vectorizer.fit_transform(movies["genres_text"])
genre_similarity = cosine_similarity(genre_matrix)

```

13.4. Collaborative Filtering

```

from scipy.sparse import csr_matrix
from sklearn.metrics.pairwise import cosine_similarity

user_codes = ratings["userId"].astype("category").cat.codes
movie_codes = ratings["movieId"].astype("category").cat.codes

utility = csr_matrix(
    (ratings["rating"], (user_codes, movie_codes))
)

item_user_matrix = utility.T
item_similarity = cosine_similarity(item_user_matrix, dense_output=False)

```

14. Tiêu chí hoàn thành

Dự án hoàn thành khi:

- Có tài liệu mô tả rõ 3 phương pháp.
- Có pipeline xử lý `movies.csv` và `ratings.csv`.
- Simple Recommender trả được top phim phổ biến/chất lượng cao.
- Content-based Recommender trả được phim cùng genre với phim đầu vào.
- Collaborative Filtering trả được phim cá nhân hóa theo `userId`.
- App không trả lại phim user đã rating trong tab Collaborative Filtering.
- Có test report cho input hợp lệ, input sai, user mới và movie không tồn tại.
- Có evaluation cho Collaborative Filtering bằng `HR@10/NDCG@10` hoặc sample evaluation.

15. Kết luận

Với dataset gồm `movies.csv` và `ratings.csv`, nhóm có đủ dữ liệu để triển khai 3 hướng gợi ý nền tảng trong Recommendation Systems.

Thứ tự làm hợp lý là:

```
Simple Recommender
-> Content-based Recommender
-> Collaborative Filtering
-> Demo UI/API
-> Evaluation
```

Simple Recommender giúp có baseline và xử lý user mới. Content-based Recommender khai thác `genres` để gợi ý phim tương tự. Collaborative Filtering tận dụng `ratings.csv` để cá nhân hóa gợi ý theo từng user. Đây là scope vừa đủ rõ, đúng với dữ liệu, và khả thi trong 2 tuần cho nhóm 5 người.